

Databricks — Beginner to Expert (Dev + Admin)

Step-by-step: develop and administer Databricks, with sample Parquet practice

clusters · Delta · medallion · MERGE · streaming · Unity Catalog · cluster policies · cost · security

A complete, step-by-step path to develop **and** administer Databricks. Uses a sample e-commerce dataset throughout (orders / customers / products). Pairs with the runnable notebook `databricks_de_practice.py` and `sample_data/` (which generate the **Parquet** files you practice on).

How the Parquet files are made: Parquet is binary, so you create it inside Databricks in one line (`df.write.parquet(...)`). Part 1, Step 4 does exactly this — that's your sample Parquet.

PART 0 — What Databricks is & how it's built

Databricks is a managed **lakehouse** platform on top of **Apache Spark** + **Delta Lake**, available on AWS/Azure/GCP. It combines a data lake's cheap storage with a warehouse's reliability and speed.

Architecture (know this):

- **Control plane** (managed by Databricks): the web UI, notebooks, job scheduler, cluster manager, and metadata.
- **Data plane** (in *your* cloud account): the **clusters** (VMs) that run Spark and your **storage** (S3/ADLS). Your data stays in your account.
- **Cluster** = a driver + workers (executors) that run your code.
- **DBFS** = a filesystem abstraction over cloud storage.
- **Unity Catalog** = the governance layer (catalogs → schemas → tables, with permissions + lineage).

Key objects: Workspace, Notebooks, Clusters, Jobs/Workflows, SQL Warehouses, Repos, Delta tables, Unity Catalog.

DEVELOPMENT

PART 1 — BEGINNER: get started

Step 1 — Sign up. Use **Databricks Community Edition** (free) or a cloud workspace.

Step 2 — Create a cluster. Compute → Create cluster → pick a runtime (LTS, e.g., 14.x) → Create. A cluster = driver + workers that run your code. (Community Edition gives a small single cluster.)

Step 3 — Create/import a notebook. Workspace → Create → Notebook (Python), attach the cluster. Notebooks mix Python, SQL, Scala, R via magics: `%python`, `%sql`, `%md`, `%fs`, `%sh`.

Step 4 — Create sample Parquet data. Build DataFrames and write Parquet:

```
from pyspark.sql import functions as F, Window
import random; random.seed(7)
REGIONS=["West","East","South","Central"]; CATS=["Electronics","Home","Toys","Sports","Books"]
customers=[(i,f"Customer {i}",random.choice(REGIONS) if random.random()>0.05 else None) for i in
range(1,201)]
products=[(i,f"Product {i}",random.choice(CATS),round(random.uniform(5,500),2)) for i in
range(1,51)]
orders=[]; oid=100000
for _ in range(10000):
    oid+=1; day=random.randint(1,40); qty=random.randint(1,5);
price=round(random.uniform(5,500),2)
    status=random.choices(["completed","cancelled","pending"],weights=[78,14,8])[0]
    amt=round(qty*price,2) if random.random()>0.03 else None
    orders.append((oid,random.randint(1,210),random.randint(1,50),qty,amt,status,f"2026-07-
{day:02d} 10:00:00"))
orders += random.sample(orders,150) # duplicates
spark.createDataFrame(customers,
["customer_id","name","region"]).write.mode("overwrite").parquet("/tmp/de/raw/customers")
spark.createDataFrame(products,
["product_id","product_name","category","unit_price"]).write.mode("overwrite").parquet("/tmp/de/
raw/products")
spark.createDataFrame(orders,
["order_id","customer_id","product_id","quantity","amount","status","order_ts"]).write.mode("ove
rwrite").parquet("/tmp/de/raw/orders")
display(dbutils.fs.ls("/tmp/de/raw/orders")) # your .parquet files
```

Step 5 — Read & explore Parquet.

```
orders = spark.read.parquet("/tmp/de/raw/orders")
orders.printSchema(); print(orders.count())
display(orders.limit(10))
display(orders.groupBy("status").count())
```

Step 6 — Basic DataFrame + SQL.

```
orders.select("order_id","amount").filter("amount > 100").show()
orders.createOrReplaceTempView("orders")
spark.sql("SELECT status, COUNT(*) FROM orders GROUP BY status").show()
```

Or `%sql` in a cell: `SELECT * FROM orders LIMIT 10;`

PART 2 — DEVELOPMENT: Intermediate

Transformations & joins.

```
customers = spark.read.parquet("/tmp/de/raw/customers")
enriched = orders.join(F.broadcast(customers), "customer_id")      # broadcast small table
by_region = (enriched.filter("status='completed'")
              .groupBy("region").agg(F.round(F.sum("amount"),2).alias("revenue")))
display(by_region)
```

Delta Lake — why & how. Delta = Parquet + a transaction log. It adds **ACID transactions, MERGE/upserts, time travel, schema enforcement/evolution, and OPTIMIZE**. Write Delta instead of Parquet for tables you update or query repeatedly.

```
by_region.write.format("delta").mode("overwrite").save("/tmp/de/gold/revenue_by_region")
spark.sql("CREATE TABLE IF NOT EXISTS revenue_by_region USING DELTA LOCATION
'/tmp/de/gold/revenue_by_region'")
```

Medallion architecture (bronze → silver → gold).

- **Bronze:** raw ingested (the Parquet from Part 1).
- **Silver:** cleaned/validated (filter completed, cast, dedup, referential integrity).

```
w = Window.partitionBy("order_id").orderBy(F.col("order_ts").desc())
silver = (orders.withColumn("amount", F.col("amount").cast("double"))
    .filter((F.col("status")=="completed") & (F.col("amount")>0))
    .withColumn("rn", F.row_number().over(w)).filter("rn=1").drop("rn")
    .join(F.broadcast(customers.select("customer_id")), "customer_id")
    .withColumn("order_date", F.to_date("order_ts")))
silver.write.format("delta").mode("overwrite").partitionBy("order_date").save("/tmp/de/silver/orders")
```

- **Gold:** business tables (star schema: fact + dimensions), aggregates for BI.

MERGE / upsert (incremental & CDC).

```
from delta.tables import DeltaTable
tgt = DeltaTable.forPath(spark, "/tmp/de/silver/orders")
(tgt.alias("t").merge(updates.alias("s"), "t.order_id = s.order_id")
    .whenMatchedUpdateAll().whenNotMatchedInsertAll().execute())
```

SCD Type 2 (track history). MERGE that closes the current row and inserts a new version:

```
MERGE INTO dim_customer t
USING staged_changes s ON t.customer_id = s.customer_id AND t.is_current = true
WHEN MATCHED AND t.region <> s.region THEN
    UPDATE SET t.is_current = false, t.end_date = current_date()
WHEN NOT MATCHED THEN
    INSERT (customer_id, region, start_date, end_date, is_current)
    VALUES (s.customer_id, s.region, current_date(), null, true);
```

Window functions.

```
w = Window.partitionBy("customer_id").orderBy(F.col("amount").desc())
top3 = silver.withColumn("rn", F.row_number().over(w)).filter("rn<=3")
```

Jobs & Workflows (scheduling). Workflows → Create Job → add a task (notebook/script) → set a schedule/trigger, retries, dependencies, and a **job cluster** (spins up per run — cheaper). Chain tasks into a DAG.

Auto Loader (incremental file ingestion). Efficiently ingest new files as they arrive:

```
(spark.readStream.format("cloudFiles")
  .option("cloudFiles.format","parquet").schema(schema)
  .load("/tmp/de/raw/orders")
  .writeStream.option("checkpointLocation","/tmp/de/_ckpt/orders")
  .trigger(availableNow=True).toTable("bronze_orders"))
```

Delta Live Tables (DLT). Declarative pipelines: define tables + **quality expectations**; Databricks manages incremental refresh and lineage.

```
import dlt
@dlt.table
@dlt.expect_or_drop("valid_amount", "amount > 0")
def silver_orders():
    return dlt.read("bronze_orders").filter("status='completed'")
```

PART 3 — DEVELOPMENT: Advanced

Performance tuning (the interview meat).

- **Partitioning:** partition Delta/Parquet by a filter column (usually date) → pruning.
- **OPTIMIZE + Z-ORDER:** `OPTIMIZE tbl ZORDER BY (customer_id)` — compact small files + data skipping on a frequently-filtered column.
- **Broadcast joins:** `F.broadcast(small_df)` to avoid shuffling the big table.
- **Data skew:** salting / AQE skew handling; isolate hot keys.
- **Caching:** `df.cache()` when reused; `unpersist()` after.
- **AQE (Adaptive Query Execution):** on by default — runtime shuffle coalescing, skew, join switching.
- **Photon:** Databricks' vectorized C++ engine — faster SQL/DataFrame; enable on the cluster.
- **Avoid Python UDFs** (opaque + serialization) — use built-ins or pandas UDFs.
- Read `df.explain(True)` — look for `Exchange` (shuffle) and join type.

Structured Streaming. `readStream` → transform → `writeStream` with a **checkpoint** for exactly-once; sinks include Delta tables. Combine with Auto Loader for file streams.

Unity Catalog (developer view). Reference tables as `catalog.schema.table`; use `GRANT` /lineage; access data by name, not path.

dbt on Databricks. Point dbt at a SQL warehouse/cluster; build models as Delta tables with tests + docs (the transformation layer on top of the lakehouse).

Testing & CI/CD.

- **Repos** — Git-backed notebooks/code; PR workflow.

- **Databricks Asset Bundles** — define jobs/pipelines/clusters as code (YAML) and deploy across dev/staging/prod via CI.
 - **Unit tests** for transformation functions (`pytest`), plus DLT/dbt data-quality tests in CI.
-

ADMINISTRATION

PART 4 — Databricks Administration

Account vs Workspace admin. The **account** manages users, workspaces, Unity Catalog metastore, and billing. **Workspace admins** manage clusters, jobs, and workspace access.

Users, groups, access. Add users; use **groups** for permissions; sync from your IdP via **SCIM**; enable **SSO/SAML**. Assign entitlements (workspace access, cluster create, SQL access).

Unity Catalog (governance) — set up.

- Create a **metastore** (one per region) and attach workspaces.
- Hierarchy: **metastore** → **catalog** → **schema** → **table/view**.
- **Grants:** `GRANT SELECT ON TABLE sales.public.orders TO group_analysts;` — table/column/row level security (dynamic views / row filters + column masks).
- **Lineage & audit** are automatic; a central place for discovery.

Cluster management.

- **All-purpose clusters** (interactive dev) vs **job clusters** (spin up per job — cheaper, isolated).
- **Cluster policies:** admin-defined rules that constrain cluster size/type/auto-termination to control cost and standardize config.
- **Pools:** pre-warmed instances to reduce cluster start time.
- **Autoscaling** (min/max workers) + **auto-termination** (shut idle clusters — critical for cost).
- Choose instance types by workload (memory- vs compute-optimized); enable **Photon**.

SQL Warehouses (for BI/analysts). Serverless / Pro / Classic SQL endpoints that run SQL for dashboards/BI tools. Right-size, set auto-stop, and use serverless for instant, elastic compute.

Secrets management. Store credentials in **secret scopes** (Databricks-backed or KMS/Key Vault-backed); read with `dbutils.secrets.get(scope, key)`. Never hard-code secrets.

Security.

- **IAM roles / instance profiles** (AWS) so clusters access S3 with scoped permissions.
- **Encryption** at rest (KMS) + in transit (TLS); optional customer-managed keys.
- **Network:** deploy in your VPC; private subnets; no public data access; IP access lists.
- **Credential passthrough / Unity Catalog** for per-user data access.

Cost management (DBUs). Databricks bills in **DBUs** (usage units) + your cloud VM cost. Control it with: **auto-termination**, **cluster policies**, **job clusters** over all-purpose, **spot/preemptible** workers, **autoscaling**, **serverless** where it fits, and **tags** for cost attribution. Monitor with system tables / usage dashboards.

Monitoring & observability. **Spark UI** (jobs/stages/tasks/shuffle), cluster metrics, **job run history** + **alerts** (email/webhook on failure), and **audit logs** (who did what). Set alerts on job failures and long runtimes.

Reliability/DR. Delta gives ACID + time travel (recover from bad writes). Replicate critical data/config across regions; version everything as code (bundles) for reproducibility.

EXPERT — Best practices & pitfalls

Development best practices

- Medallion zones; Delta everywhere; partition + `OPTIMIZE` /Z-ORDER on big tables.
- Incremental (Auto Loader / MERGE), **idempotent** writes, checkpoints for streaming.
- Data-quality expectations (DLT/dbt) that fail bad data.
- Avoid UDFs; broadcast small tables; watch skew; read the plan.
- Git Repos + Asset Bundles + tests in CI/CD.

Administration best practices

- **Auto-termination + cluster policies** (biggest cost lever).
- **Unity Catalog** for governance; least-privilege grants; groups not individuals.
- Secrets in scopes; instance profiles for storage; encryption + VPC.
- Job clusters for scheduled work; pools for fast starts; right-size + Photon.
- Tag resources; monitor DBU usage; alert on failures.

Common issues → fixes

Issue	Fix
Cluster costs high	Auto-termination, cluster policies, job clusters, spot workers
Job slow / big shuffle	Broadcast small tables, partition/Z-ORDER, fix skew, Photon
Small-files problem	<code>OPTIMIZE</code> (compaction), sensible partitioning
"Table not found" / access denied	Unity Catalog grants; use <code>catalog.schema.table</code>
Streaming duplicates	Checkpointing + idempotent MERGE
Secrets in code	Move to secret scopes

Quick roadmap (beginner → expert)

1. **Beginner:** cluster + notebook, read/write Parquet, DataFrame + SQL basics.
2. **Intermediate:** Delta, medallion, MERGE, windows, Jobs, Auto Loader, DLT.
3. **Advanced dev:** performance tuning, streaming, Unity Catalog, CI/CD (Repos + bundles).
4. **Admin:** users/groups/UC governance, cluster policies/pools, SQL warehouses, secrets, security, **cost control**, monitoring.
5. **Expert:** best practices + troubleshooting across dev and admin; automate everything as code.

(Run the companion notebook `databricks_de_practice.py` to do steps 1–3 hands-on on real Parquet.)